

1:50-2:15pm, Monday, March 9, 2026.

1. **Dynamic Programming tables.** In each case, we are given a recursion for a two-dimensional array  $A[1..n, 1..n]$ . Either write two for-loops that compute all values of the array; or prove that it is impossible to compute all values of the array using the formula.

(a)

$$A[i, j] = \begin{cases} i + j & \text{if } i = 1, i = n, \text{ or } j = 1 \\ A[i - 1, j] + A[i + 1, j - 1], & \text{otherwise.} \end{cases}$$

**Answer.** We can compute the entries column-by-column:

```

begin
  for j = 1 to n do
    for i = 1 to n do
      if i = 1 or i = n or j = 1 then
        A[i, j] ← i + j
      else
        A[i, j] ← A[i - 1, j] + A[i + 1, j - 1]
    end for
  end for
end

```

(b)

$$A[i, j] = \begin{cases} i + j & \text{if } i = 1, i = 2, i = n, j = 1, \text{ or } j = n \\ A[i - 2, j + 1] + A[i + 1, j - 1] & \text{otherwise.} \end{cases}$$

**Answer.** We can compute the entries diagonal-by-diagonal:

```

begin
  for i = 1 to n do
    for ℓ = 0 to i - 1 do
      if i - ℓ = 1 or i - ℓ = 2 or i - ℓ = n or ℓ + 1 = 1 or ℓ + 1 = n then
        A[i - ℓ, ℓ + 1] ← i + j
      else
        A[i - ℓ, ℓ + 1] ← A[i + ℓ - 1, ℓ + 2] + A[i + ℓ + 1, ℓ]
      end if
    end for
    for j = 2 to n do
      for ℓ = 0 to n - j do
        if n - ℓ = 1 or n - ℓ = 2 or n - ℓ = n or j + ℓ = 1 or j + ℓ = n then
          A[n - ℓ, j + ℓ] ← i + j
        else
          A[n - ℓ, j + ℓ] ← A[n - ℓ - 2, j + ℓ + 1] + A[n - ℓ + 1, j + ℓ - 1]
        end if
      end for
    end for
  end for
end

```

2. **Greedy Knapsack.** Assume that the values  $V[1..n]$  are arbitrary, every weight in  $W[1..n]$  is 1 or 2. Charlie suggests the following algorithm:

Sort the items by nonincreasing *value per unit weight*  $U[i] = V[i]/W[i]$ .

For  $i = 1$  to  $n$ , add item  $i$  to the bag if the total weight remains below capacity.

Either prove that this algorithm always returns OPT, or find a counterexample.

**Answer.** Consider the input  $V[1, 2] = [2, 3]$ ,  $W[1, 2] = [1, 2]$  with capacity  $C = 2$ . Then  $U[1, 2] = [2, \frac{3}{2}]$ . Clearly, we cannot choose both items, because that would be over capacity. So any solution chooses at most one item. Charlie's algorithm chooses the first item at value  $V[1] = 2$ , however OPT is the 2nd item at value  $V[2] = 3$ .

3. **Greedy Interval Scheduling.** Recall that in the interval scheduling problem, we are given  $n$  requests/intervals  $(S[1], F[1]), \dots, (S[n], F[n])$  with values  $V[1..n]$ , and we need to find a conflict-free subset of maximum value. In class, we discussed a greedy algorithm for the special case where all intervals have unit value.

Now suppose that the values  $V[1..n]$  are arbitrary, the start times  $S[1..n]$  are different odd integers, and every interval has length 2 (that is,  $F[i] = S[i] + 2$ ). Does the greedy algorithm work in this case? Either prove that it does or find a counterexample.

**Answer.** Since the difference between any two odd integers is 2 or more, then there is no conflict between any pair of input intervals. The optimal solution accepts all intervals, and so does the greedy algorithm. Therefore the greedy algorithm always returns the optimal solution.